Mục lục

1 Giới thiệu bài toán **3**

2 Mô hình tối ưu và dữ liệu **4**
2.1 Mô hình Hồi quy Logistic 4
2.1.1 Tổng quan về mô hình 4
2.1.2 Công thức toán học 4
2.1.3 Cơ sở xác suất và Ước lượng hợp lý cực đại (MLE) 4
2.2 Dữ liệu thực nghiệm 5
2.2.1 Tổng quan về bộ dữ liệu 5
2.2.2 Ý nghĩa của các biến 5
2.2.3 Tiền xử lý dữ liệu 6

3 Thuật toán tối ưu **6**
3.1 Tính toán Gradient (Đạo hàm bậc nhất) 6
3.1.1 Lựa chọn biến thể: Batch Gradient Descent 7
3.1.2 Cơ sở lý thuyết và thuật toán 7
3.1.3 Áp dụng vào bài toán hồi quy Logistic 9
3.1.4 Code triển khai Python 10
3.2 Phương pháp Newton 10
3.2.1 Cơ sở lý thuyết và thuật toán 10
3.2.2 Áp dụng vào bài toán hồi quy Logistic 11
3.2.3 Code triển khai Python 12
3.3 So sánh lý thuyết giữa Gradient Descent và phương pháp Newton 12

4 Kết quả thực nghiệm và phân tích **13**
4.1 Thí nghiệm 1: So sánh tốc độ hội tụ cơ bản 13
4.2 Thí nghiệm 2: Khảo sát sự nhạy cảm của GD với Learning Rate 14
4.3 Thí nghiệm 3: Khảo sát ảnh hưởng của chuẩn hóa dữ liệu 15

5 Kết luận **15**
5.1 Kết quả đạt được 15
5.2 Kết luận và Ý nghĩa thực tiễn 16
5.3 Hạn chế 16

Tài liệu tham khảo **17**

1 Giới thiệu bài toán

Trong lĩnh vực học máy, bài toán phân loại là một trong những tác vụ nền tảng và có ứng dụng rộng rãi nhất. Trong số các mô hình dùng để giải quyết bài toán này, Hồi quy Logistic (Logistic Regression) nổi bật như một phương pháp kinh điển, hiệu quả và có khả năng diễn giải cao, đặc biệt trong các lĩnh vực như y sinh, tài chính và kinh tế lượng. Quá trình huấn luyện một mô hình hồi quy Logistic thực chất quy về một bài toán tối ưu hóa: tìm ra bộ tham số (trọng số) của mô hình sao cho hàm mất mát trên tập dữ liệu huấn luyện đạt giá trị nhỏ nhất.

Vấn đề trọng tâm của báo cáo này xuất phát từ một đặc tính quan trọng của bài toán hồi quy Logistic. Mặc dù hàm mất mát Binary Cross-Entropy là một hàm lồi – một đặc tính lý tưởng đảm bảo sự tồn tại của một điểm cực tiểu toàn cục duy nhất – nó lại không có nghiệm chính xác như trong bài toán hồi quy tuyến tính với sai số bình phương nhỏ nhất. Điều này đặt ra một câu hỏi then chốt: **Làm thế nào để tìm ra bộ trọng số tối ưu một cách hiệu quả khi không thể giải trực tiếp?**

Để trả lời câu hỏi này, việc lựa chọn một thuật toán tối ưu hóa trở thành một quyết định có ảnh hưởng sâu sắc đến hiệu năng, tốc độ và tính khả thi của quá trình huấn luyện. Đồ án này tiến hành khảo sát sâu về hai họ thuật toán tiêu biểu, đại diện cho hai trường phái tiếp cận khác nhau trong lĩnh vực tối ưu hóa:

- **Gradient Descent (GD):** Là đại diện tiêu biểu cho các **phương pháp bậc một (first-order methods)**. Thuật toán này chỉ sử dụng thông tin về đạo hàm bậc nhất (gradient) để xác định hướng đi. Ưu điểm của nó là sự đơn giản, chi phí tính toán mỗi vòng lặp thấp và là nền tảng cho nhiều thuật toán hiện đại được dùng trong học sâu.

- **Phương pháp Newton:** Là đại diện cho các **phương pháp bậc hai (second-order methods)**. Ngoài gradient, thuật toán này còn tận dụng thông tin về đạo hàm bậc hai (ma trận Hessian) để mô tả độ cong của bề mặt hàm mất mát, từ đó thực hiện những bước nhảy hiệu quả hơn để tiến đến điểm tối ưu.

Việc so sánh hai thuật toán này làm nổi bật một sự đánh đổi kinh điển trong lĩnh vực tối ưu hóa: sự đánh đổi giữa tốc độ hội tụ lý thuyết (số vòng lặp) và chi phí tính toán trên mỗi vòng lặp. Phương pháp Newton được kỳ vọng sẽ hội tụ với số vòng lặp ít hơn rất nhiều, nhưng mỗi vòng lặp lại đòi hỏi chi phí tính toán và nghịch đảo ma trận Hessian vô cùng tốn kém. Ngược lại, Gradient Descent có chi phí mỗi vòng lặp rẻ hơn nhưng thường cần nhiều vòng lặp hơn để hội tụ.

Do đó, báo cáo này mang ý nghĩa thực tiễn quan trọng. Bằng cách thực hiện các thí nghiệm so sánh kỹ lưỡng trên một bộ dữ liệu thực tế (Heart Disease UCI), kết quả của đồ án sẽ cung cấp một kết quả hữu ích, giúp trả lời câu hỏi: *Trong bối cảnh nào thì sự đơn giản của Gradient Descent là đủ tốt, và khi nào thì sự phức tạp về mặt tính toán của Phương pháp Newton mang lại lợi ích xứng đáng?* Thông qua việc phân tích tốc độ hội tụ, sự ảnh hưởng của siêu tham số và tác động của việc chuẩn hóa dữ liệu, đồ án sẽ đưa ra một cái nhìn toàn diện về hiệu năng của hai phương pháp này trong thực tế.

2 Mô hình tối ưu và dữ liệu

2.1 Mô hình Hồi quy Logistic

2.1.1 Tổng quan về mô hình

Hồi quy Logistic là một mô hình học máy có giám sát được sử dụng cho bài toán phân loại nhị phân. Mục tiêu của mô hình là dự đoán xác suất một đối tượng thuộc về một trong hai lớp (ví dụ, lớp 1 hoặc lớp 0) dựa trên một tập hợp các thuộc tính đầu vào.

Mô hình này hoạt động dựa trên hai thành phần chính:

1. **Một hàm tuyến tính:** Tính toán một điểm số dựa trên tổ hợp tuyến tính của các thuộc tính đầu vào và các trọng số tương ứng.
2. **Hàm Sigmoid (Hàm Logistic):** Chuyển đổi điểm số tuyến tính đó thành một giá trị xác suất trong khoảng (0, 1).

2.1.2 Công thức toán học

Giả sử chúng ta có một tập dữ liệu gồm N mẫu. Với mỗi mẫu thứ i, ta có một vector thuộc tính $\mathbf{x}_i$ và một nhãn y_i.

- **Vector thuộc tính (Biến đầu vào):** $\mathbf{x}_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,d}]^T$ là một vector cột chứa d giá trị thuộc tính của mẫu thứ i. Để đơn giản hóa, ta thường thêm một chiều bias $x_{i,0} = 1$ vào vector này, do đó $\mathbf{x}_i$ có kích thước $(d+1) \times 1$.
- **Vector trọng số (Tham số):** $\mathbf{w} = [w_0, w_1, \ldots, w_d]^T$ là vector cột chứa các tham số của mô hình cần được học. w_0 là hệ số chặn (bias), và w_j (với $j > 0$) là trọng số tương ứng với thuộc tính thứ j.
- **Nhãn (Biến mục tiêu):** $y_i \in \{0, 1\}$ là nhãn thực tế của mẫu thứ i.

Xác suất dự đoán p_i rằng mẫu thứ i thuộc về lớp 1 được tính như sau:

$$p_i = P(y_i = 1|\mathbf{x}_i; \mathbf{w}) = \sigma(\mathbf{w}^T\mathbf{x}_i) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}_i}} \tag{1}$$

trong đó $\sigma(\cdot)$ là hàm Sigmoid.

2.1.3 Cơ sở xác suất và Ước lượng hợp lý cực đại (MLE)

Thay vì áp dụng trực tiếp Hồi quy tuyến tính vào bài toán phân loại (điều này có thể dẫn đến các giá trị dự đoán nằm ngoài khoảng $[0, 1]$), ta tiếp cận bài toán dưới góc độ xác suất.

Giả sử biến mục tiêu y tuân theo phân phối Bernoulli. Khi đó, xác suất xảy ra của các lớp với bộ tham số θ (trong báo cáo này ký hiệu là $\mathbf{w}$) được định nghĩa như sau:

$$P(y = 1|\mathbf{x}; \mathbf{w}) = h_{\mathbf{w}}(\mathbf{x})$$
$$P(y = 0|\mathbf{x}; \mathbf{w}) = 1 - h_{\mathbf{w}}(\mathbf{x})$$

Công thức tổng quát cho xác suất của một mẫu dữ liệu $(\mathbf{x}, y)$ có thể viết gọn là:

$$P(y|\mathbf{x}; \mathbf{w}) = (h_{\mathbf{w}}(\mathbf{x}))^y(1 - h_{\mathbf{w}}(\mathbf{x}))^{1-y} \tag{2}$$

Giả sử N mẫu dữ liệu huấn luyện được sinh ra độc lập. Hàm hợp lý (Likelihood function) $L(\mathbf{w})$ là xác suất đồng thời của toàn bộ dữ liệu quan sát được:

$$L(\mathbf{w}) = \prod_{i=1}^{N} P(y^{(i)}|\mathbf{x}^{(i)}; \mathbf{w}) = \prod_{i=1}^{N} (h_{\mathbf{w}}(\mathbf{x}^{(i)}))^{y^{(i)}} (1 - h_{\mathbf{w}}(\mathbf{x}^{(i)}))^{1-y^{(i)}} \tag{3}$$

Để tìm bộ tham số tốt nhất, ta cần tối đa hóa hàm hợp lý này. Để thuận tiện cho việc tính toán đạo hàm, ta lấy logarit tự nhiên hai vế để chuyển từ tích sang tổng, thu được Hàm log-hợp lý (Log-likelihood):

$$\ell(\mathbf{w}) = \log L(\mathbf{w}) = \sum_{i=1}^{N} \left[y^{(i)} \log(h_{\mathbf{w}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right] \tag{4}$$

Trong tối ưu hóa, chúng ta thường giải bài toán tối thiểu hóa. Do đó, việc tối đa hóa hàm Log-likelihood tương đương với việc tối thiểu hóa hàm đối ngẫu của nó. Ta định nghĩa hàm mất mát $J(\mathbf{w})$ (Binary Cross-Entropy) chính là trung bình của âm Log-likelihood:

$$J(\mathbf{w}) = -\frac{1}{N} \ell(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^{N} \left[y^{(i)} \log(h_{\mathbf{w}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right] \tag{5}$$

Đây chính là hàm mục tiêu mà cả Gradient Descent và phương pháp Newton sẽ cùng tối thiểu hóa.

2.2 Dữ liệu thực nghiệm

2.2.1 Tổng quan về bộ dữ liệu

Báo cáo sử dụng bộ dữ liệu công khai **Heart Disease UCI**, được thu thập từ Cleveland Clinic Foundation. Đây là một bộ dữ liệu kinh điển trong lĩnh vực học máy, thường được dùng làm bộ dữ liệu tham chiếu để so sánh hiệu năng của các thuật toán phân loại.

- **Số lượng mẫu:** 303 bệnh nhân.
- **Số lượng thuộc tính:** 13 thuộc tính đầu vào và 1 biến mục tiêu (‘target‘).

2.2.2 Ý nghĩa của các biến

Các biến trong bộ dữ liệu này có ý nghĩa lâm sàng rõ ràng, bao gồm:

1. **age:** Tuổi của bệnh nhân.
2. **sex:** Giới tính (1 = nam; 0 = nữ).
3. **cp:** Loại đau ngực (chest pain type).
4. **trestbps:** Huyết áp lúc nghỉ (resting blood pressure).
5. **chol:** Nồng độ cholesterol trong huyết thanh.
6. **fbs:** Lượng đường trong máu lúc đói > 120 mg/dl (1 = true; 0 = false).
7. **restecg:** Kết quả điện tâm đồ lúc nghỉ.

8. **thalach:** Nhịp tim tối đa đạt được.
9. **exang:** Đau thắt ngực do gắng sức (1 = có; 0 = không).
10. **oldpeak:** Đoạn ST chênh xuống do gắng sức so với lúc nghỉ.
11. **slope:** Độ dốc của đoạn ST lúc gắng sức đỉnh điểm.
12. **ca:** Số lượng mạch máu chính được nhuộm màu bằng phương pháp soi huỳnh quang.
13. **thal:** Tình trạng thalassemia (một loại bệnh về máu).
14. **target (biến mục tiêu y):** Có bệnh tim hay không (1 = có; 0 = không).

2.2.3 Tiền xử lý dữ liệu

- **Phân chia dữ liệu thành tập train và tập test:** Dữ liệu được chia ngẫu nhiên thành hai phần: 80% cho tập huấn luyện và 20% cho tập kiểm tra. Tập huấn luyện được dùng để huấn luyện cho mô hình, còn tập kiểm tra được dùng để đánh giá khách quan hiệu năng thực sự của mô hình sau khi đã huấn luyện xong.
- **Chuẩn hóa Dữ liệu:** Các thuộc tính đầu vào được chuẩn hóa để có trung bình bằng 0 và phương sai bằng 1. Bước này là tối quan trọng, đặc biệt đối với các thuật toán như Gradient Descent, vì nó giúp đảm bảo rằng tất cả các thuộc tính đều có cùng một thang đo, từ đó cải thiện tốc độ và sự ổn định của quá trình hội tụ.

3 Thuật toán tối ưu

Để áp dụng các thuật toán tối ưu dựa trên đạo hàm (như Gradient Descent và Newton), bước quan trọng nhất là tính toán vector gradient (đạo hàm bậc nhất) và ma trận Hessian (đạo hàm bậc hai) của hàm mất mát.

3.1 Tính toán Gradient (Đạo hàm bậc nhất)

Trước hết, ta xét đạo hàm của hàm Sigmoid $g(z) = \frac{1}{1+e^{-z}}$. Một tính chất quan trọng của hàm này là:

$$g'(z) = g(z)(1 - g(z))$$

Áp dụng quy tắc chuỗi để tính đạo hàm riêng của hàm mất mát $J(\mathbf{w})$ theo tham số w_j:

$$\begin{aligned}
\frac{\partial J(\mathbf{w})}{\partial w_j} &= -\frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)}\frac{1}{h_{\mathbf{w}}(\mathbf{x}^{(i)})} - (1-y^{(i)})\frac{1}{1-h_{\mathbf{w}}(\mathbf{x}^{(i)})}\right)\frac{\partial h_{\mathbf{w}}(\mathbf{x}^{(i)})}{\partial w_j} \\
&= -\frac{1}{N}\sum_{i=1}^{N}\left(\frac{y^{(i)} - h_{\mathbf{w}}(\mathbf{x}^{(i)})}{h_{\mathbf{w}}(\mathbf{x}^{(i)})(1-h_{\mathbf{w}}(\mathbf{x}^{(i)}))}\right)\underbrace{h_{\mathbf{w}}(\mathbf{x}^{(i)})(1-h_{\mathbf{w}}(\mathbf{x}^{(i)}))x_j^{(i)}}_{\text{Đạo hàm của hàm hợp sigmoid}} \\
&= \frac{1}{N}\sum_{i=1}^{N}(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})x_j^{(i)}
\end{aligned}$$

Viết dưới dạng ma trận cho toàn bộ tập dữ liệu, ta có vector Gradient:

$$\nabla J(\mathbf{w}) = \frac{1}{N}\mathbf{X}^T(\mathbf{h} - \mathbf{y}) \tag{6}$$

Trong đó $\mathbf{h} = \sigma(\mathbf{X}\mathbf{w})$ là vector xác suất dự đoán. Công thức gradient này chính là nền tảng để xây dựng quy tắc cập nhật cho cả hai thuật toán dưới đây.

3.1.1 Lựa chọn biến thể: Batch Gradient Descent

Trước khi đi vào chi tiết, cần làm rõ rằng phương pháp Gradient Descent được triển khai trong báo cáo này là biến thể **Batch Gradient Descent**. Điều này có nghĩa là tại mỗi bước cập nhật, vector gradient được tính toán dựa trên **toàn bộ** tập dữ liệu huấn luyện. Biến thể này khác với Stochastic Gradient Descent (SGD) - tính gradient trên một mẫu duy nhất, và Mini-batch Gradient Descent - tính trên một lô nhỏ dữ liệu.

Việc lựa chọn Batch GD là một quyết định có chủ đích và hoàn toàn phù hợp với mục tiêu của đồ án, vì những lý do sau:

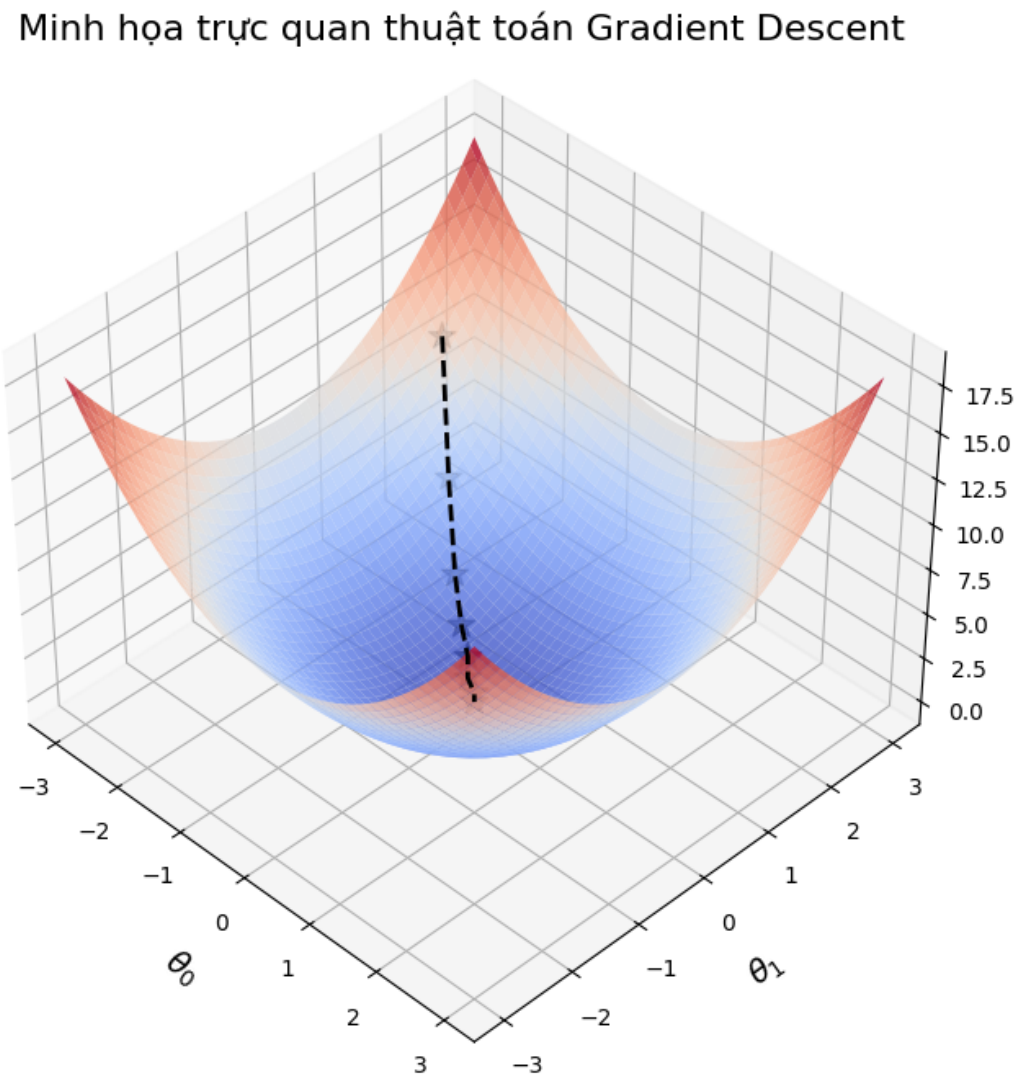
- **Quy mô dữ liệu cho phép:** Bộ dữ liệu Heart Disease UCI chỉ gồm 303 mẫu. Với quy mô nhỏ này, chi phí tính toán gradient trên toàn bộ dữ liệu là không đáng kể, loại bỏ được nhược điểm lớn nhất của Batch GD là tốc độ chậm trên dữ liệu lớn.
- **Tạo ra một sự so sánh ngang bằng:** Phương pháp Newton được khảo sát cũng là một thuật toán "batch"vì nó yêu cầu tính toán ma trận Hessian trên toàn bộ dữ liệu. Việc so sánh Batch GD (bậc một) với Phương pháp Newton (bậc hai) đảm bảo rằng cả hai thuật toán đều đang hoạt động trên cùng một lượng thông tin tại mỗi bước lặp.
- **Lý tưởng cho việc kiểm chứng lý thuyết:** Vì gradient được tính trên toàn bộ dữ liệu, hướng đi của thuật toán là chính xác và không bị nhiễu do lấy mẫu ngẫu nhiên. Điều này tạo ra một đường cong hội tụ mượt mà, lý tưởng để phân tích và minh họa các đặc tính lý thuyết như tốc độ hội tụ tuyến tính một cách rõ ràng nhất.

3.1.2 Cơ sở lý thuyết và thuật toán

Nền tảng của phương pháp lặp Trong nhiều bài toán học máy, hàm mục tiêu $L(\mathbf{w})$ quá phức tạp để tìm nghiệm tối ưu $\mathbf{w}^* = \arg\min_{\mathbf{w}} L(\mathbf{w})$ bằng phương pháp giải tích trực tiếp. Do đó, chúng ta cần đến các thuật toán tối ưu hóa dựa trên phương pháp lặp. **Gradient Descent** (còn gọi là Batch Gradient Descent) là một trong những thuật toán nền tảng và đơn giản nhất.

Trực quan, ta có thể hình dung $L(\mathbf{w})$ là một bề mặt trong không gian nhiều chiều (thường có dạng hình cái bát đối với Hồi quy Logistic). Mục tiêu là tìm điểm thấp nhất trên bề mặt đó. Gradient Descent hoạt động bằng cách bắt đầu từ một điểm $\mathbf{w}_0$ ngẫu nhiên, sau đó liên tục cập nhật việc đi một bước nhỏ theo hướng dốc nhất xuống dưới (hướng ngược lại của gradient).

Quá trình này được minh họa cụ thể như hình dưới đây:



Hình 1: Minh họa trực quan Gradient Descent: Thuật toán bắt đầu từ vùng có giá trị hàm mất mát cao (màu đỏ) và thực hiện các bước nhảy (các dấu sao) đi xuống đáy thung lũng (màu xanh) để tìm điểm cực tiểu.

Các bước thực hiện thuật toán:

Bước 1: Chọn một tốc độ học η (learning rate), là một số dương nhỏ.

Bước 2: Khởi tạo giá trị tham số $\mathbf{w}_0$ (thường là vector không).

Bước 3: Cập nhật tất cả các tham số dựa trên gradient của hàm mất mát tính trên **toàn bộ** tập dữ liệu huấn luyện:

$$\mathbf{w}_{t+1} := \mathbf{w}_t - \eta \cdot \nabla_{\mathbf{w}} L(\mathbf{w}_t)$$

Bước 4: Lặp lại Bước 3 cho đến khi một điều kiện dừng được thỏa mãn (ví dụ: độ lớn của gradient đủ nhỏ hoặc chạy đủ số vòng lặp).

Về tốc độ học (Learning Rate) Tham số η là hằng số nhân, nhưng độ lớn thực tế của bước cập nhật ($||\eta \nabla L(\mathbf{w})||$) không phải là hằng số, mà thay đổi tùy thuộc vào độ lớn của gradient. Việc chọn η là một sự đánh đổi quan trọng:

- η quá nhỏ sẽ khiến thuật toán hội tụ rất chậm.
- η quá lớn có thể khiến các bước nhảy vượt qua điểm cực tiểu, gây ra dao động mạnh và có thể không bao giờ hội tụ (phân kỳ).

3.1.3 Áp dụng vào bài toán hồi quy Logistic

Vai trò của thuật toán: Trong bài toán Hồi quy Logistic, không tồn tại một công thức nghiệm chính xác cho bộ trọng số tối ưu $\mathbf{w}^*$. Do đó, việc sử dụng một thuật toán tối ưu lặp như Gradient Descent là **bắt buộc**. Vai trò của GD là tìm kiếm một cách lặp đi lặp lại trong không gian tham số để tìm ra bộ trọng số $\mathbf{w}$ giúp tối thiểu hóa hàm mất mát Binary Cross-Entropy. Vì hàm mất mát này là một hàm lồi, GD được đảm bảo sẽ hội tụ về điểm cực tiểu toàn cục duy nhất.

Đối với bài toán cụ thể là dự đoán bệnh tim với bộ dữ liệu Heart Disease, GD sẽ tìm ra một vector trọng số $\mathbf{w}$ sao cho mỗi thuộc tính (tuổi, huyết áp, cholesterol, v.v.) được gán một mức độ quan trọng phù hợp, giúp mô hình phân biệt tốt nhất giữa hai lớp "có bệnh tim"và "không có bệnh tim".

Công thức Gradient và Pseudo code: Để áp dụng GD, chúng ta cần tính vector gradient của hàm Binary Cross-Entropy:

$$\nabla L(\mathbf{w}) = \frac{1}{N}\mathbf{X}^T(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y}) \tag{7}$$

Quá trình này được tóm tắt trong giả thuật dưới đây.

Algorithm 1 Gradient Descent cho Hồi quy Logistic

Khởi tạo: $\mathbf{w}$, learning rate η, số vòng lặp T, ngưỡng hội tụ ϵ.
for $t = 1, 2, \ldots, T$ **do**
    Tính xác suất dự đoán: $p = \sigma(\mathbf{X}\mathbf{w})$
    Tính gradient: $\nabla L(\mathbf{w}) = \frac{1}{N}\mathbf{X}^T(p - \mathbf{y})$
    Cập nhật trọng số: $\mathbf{w} := \mathbf{w} - \eta \nabla L(\mathbf{w})$
    if $||\nabla L(\mathbf{w})|| < \epsilon$ **then**
        break
    end if
end for
Trả về: $\mathbf{w}$

3.1.4 Code triển khai Python

```python
def gradient_descent_method(X, y, learning_rate, iterations, tolerance=1e-6):
    N, d = X.shape
    w = np.zeros((d, 1))
    loss_history = []

    for i in range(iterations):
        p = sigmoid(X @ w)
        gradient = (1/N) * X.T @ (p - y)
        w = w - learning_rate * gradient
        loss_history.append(compute_loss(X, y, w))

        if np.linalg.norm(gradient) < tolerance:
            print(f"GD converged at iteration {i+1}")
            break

    return w, loss_history
```

Listing 1: Hàm Gradient Descent trong Python

3.2 Phương pháp Newton

3.2.1 Cơ sở lý thuyết và thuật toán

Từ bậc một đến bậc hai Nếu Gradient Descent chỉ sử dụng thông tin bậc một (đạo hàm riêng, tức là độ dốc), thì Phương pháp Newton là một thuật toán bậc hai, tận dụng cả thông tin bậc hai (đạo hàm riêng bậc hai, tức là độ cong của hàm mất mát). Thông tin về độ cong được chứa trong **ma trận Hessian H**.

Nền tảng toán học: Xấp xỉ Taylor bậc hai Cơ sở của phương pháp Newton là xấp xỉ hàm mất mát $L(\mathbf{w})$ tại lân cận của điểm hiện tại $\mathbf{w}_t$ bằng một đa thức Taylor bậc hai:

$$L(\mathbf{w}_t + \Delta\mathbf{w}) \approx L(\mathbf{w}_t) + \nabla L(\mathbf{w}_t)^T \Delta\mathbf{w} + \frac{1}{2}\Delta\mathbf{w}^T \mathbf{H}(\mathbf{w}_t)\Delta\mathbf{w}$$

Đây là một hàm bậc hai theo $\Delta\mathbf{w}$. Thay vì tối thiểu hóa hàm L phức tạp, ta sẽ tối thiểu hóa hàm xấp xỉ bậc hai này. Bằng cách lấy đạo hàm theo $\Delta\mathbf{w}$ và cho bằng 0, ta tìm được bước nhảy $\Delta\mathbf{w}$ tối ưu cho mô hình xấp xỉ:

$$\Delta\mathbf{w} = -[\mathbf{H}(\mathbf{w}_t)]^{-1} \nabla L(\mathbf{w}_t)$$

Đây chính là "bước nhảy Newton".

Các bước thực hiện thuật toán:

Bước 1: Khởi tạo giá trị tham số $\mathbf{w}_0$.

Bước 2: Tại mỗi bước lặp t, tính toán:

- Vector gradient $\nabla_{\mathbf{w}} L(\mathbf{w}_t)$.
- Ma trận Hessian $\mathbf{H}(\mathbf{w}_t)$.

Bước 3: Cập nhật tham số bằng cách thực hiện bước nhảy Newton:

$$\mathbf{w}_{t+1} := \mathbf{w}_t - [\mathbf{H}(\mathbf{w}_t)]^{-1} \nabla_{\mathbf{w}} L(\mathbf{w}_t)$$

Bước 4: Lặp lại Bước 2 và 3 cho đến khi hội tụ.

Một ưu điểm lớn của phương pháp này là nó không cần đến siêu tham số learning rate.

3.2.2 Áp dụng vào bài toán hồi quy Logistic

Vai trò của thuật toán: Tương tự GD, vai trò của Phương pháp Newton là tìm ra bộ trọng số tối ưu $\mathbf{w}^*$ cho bài toán Hồi quy Logistic. Tuy nhiên, bằng cách sử dụng thông tin về độ cong (Hessian), Newton có thể thực hiện những bước nhảy lớn và "thông minh"hơn trong không gian tham số. Thay vì đi từng bước nhỏ theo hướng dốc nhất, nó nhảy thẳng đến gần điểm cực tiểu của bề mặt hàm loss.

Đối với bài toán dự đoán bệnh tim, việc sử dụng Newton có thể giúp quá trình huấn luyện tìm ra bộ trọng số tối ưu với số lần cập nhật ít hơn rất nhiều, giúp tiết kiệm thời gian tính toán trong các bối cảnh mà chi phí tính Hessian là chấp nhận được.

Công thức Hessian và Pseudo code: Để áp dụng Phương pháp Newton, ta cần đạo hàm bậc hai của hàm mất mát. Bằng cách lấy đạo hàm của vector Gradient $\nabla J(\mathbf{w})$ một lần nữa theo $\mathbf{w}$, ta thu được ma trận Hessian:

$$\mathbf{H}(\mathbf{w}) = \nabla^2 J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T \mathbf{D} \mathbf{X} \tag{8}$$

Trong đó $\mathbf{D}$ là ma trận đường chéo với các phần tử $D_{ii} = p_i(1 - p_i)$, thể hiện phương sai của phân phối Bernoulli tại mỗi điểm dữ liệu.

Algorithm 2 Phương pháp Newton cho Hồi quy Logistic

Khởi tạo: $\mathbf{w}$, số vòng lặp T, ngưỡng hội tụ ϵ.
for $t = 1, 2, \ldots, T$ **do**
    Tính xác suất dự đoán: $p = \sigma(\mathbf{X}\mathbf{w})$
    Tính gradient: $\nabla L(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (p - \mathbf{y})$
    Tạo ma trận đường chéo $\mathbf{D}$ với $D_{ii} = p_i(1 - p_i)$
    Tính Hessian: $\mathbf{H} = \frac{1}{N} \mathbf{X}^T \mathbf{D} \mathbf{X}$
    Cập nhật trọng số: $\mathbf{w} := \mathbf{w} - \mathbf{H}^{-1} \nabla L(\mathbf{w})$
    if $||\nabla L(\mathbf{w})|| < \epsilon$ **then**
        break
    end if
end for
Trả về: $\mathbf{w}$

3.2.3 Code triển khai Python

```python
def newtons_method(X, y, iterations):
    N, d = X.shape
    w = np.zeros((d, 1))
    loss_history = []
    epsilon = 1e-6

    for i in range(iterations):
        p = sigmoid(X @ w).flatten()
        gradient = (1/N) * X.T @ (p.reshape(-1,1) - y)

        D = np.diag(p * (1 - p))
        Hessian = (1/N) * X.T @ D @ X

        Hessian_inv = np.linalg.inv(Hessian + epsilon * np.eye(d))
        w = w - Hessian_inv @ gradient
        loss_history.append(compute_loss(X, y, w))

        if np.linalg.norm(gradient) < 1e-6:
            print(f"Newton's method converged at {i+1}")
            break

    return w, loss_history
```

Listing 2: Hàm Phương pháp Newton trong Python

3.3 So sánh lý thuyết giữa Gradient Descent và phương pháp Newton

Sau khi mô tả riêng lẻ, việc đặt hai thuật toán cạnh nhau sẽ làm nổi bật sự đánh đổi (trade-off) kinh điển trong lĩnh vực tối ưu hóa. Bảng 1 tóm tắt các khác biệt cốt lõi.

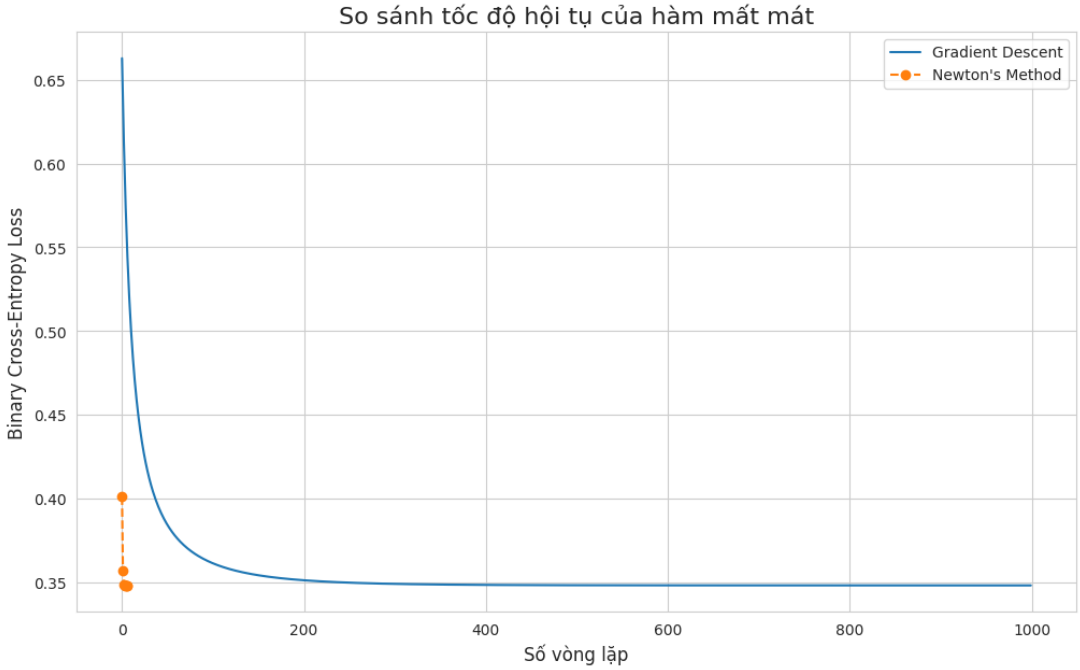
Bảng 1: So sánh các đặc tính lý thuyết của GD và Newton.

Tiêu chí	**Gradient Descent**	**Phương pháp Newton**
Bậc thông tin sử dụng	Bậc một (First-order) Chỉ dùng Gradient $\nabla L(\mathbf{w})$.	Bậc hai (Second-order) Dùng cả Gradient $\nabla L(\mathbf{w})$ và Hessian $\mathbf{H}(\mathbf{w})$.
Bản chất bước cập nhật	Đi một bước nhỏ theo hướng dốc nhất (steepest descent).	Nhảy đến điểm cực tiểu của mô hình bậc hai xấp xỉ hàm loss.
Tốc độ hội tụ	Tuyến tính (Linear) Chậm, đặc biệt khi ở gần cực tiểu.	Bậc hai (Quadratic) Cực nhanh khi ở gần cực tiểu.
Chi phí / vòng lặp	Thấp, $O(Nd)$. (Tính gradient)	Rất cao, $O(Nd^2 + d^3)$. (Tính Hessian và nghịch đảo ma trận)
Siêu tham số	Có (Learning rate η). Cần phải được tinh chỉnh cẩn thận.	Không (trong dạng cơ bản). Tự động xác định bước nhảy.
Độ nhạy cảm	Nhạy cảm với thang đo của dữ liệu.	Ít nhạy cảm hơn (affine invariant).

Tổng kết sự đánh đổi: Rõ ràng, không có thuật toán nào tốt nhất một cách tuyệt đối. Gradient Descent đơn giản và có chi phí tính toán cho mỗi vòng lặp thấp, nhưng cần nhiều vòng lặp và sự tinh chỉnh cẩn thận. Phương pháp Newton hội tụ cực nhanh về mặt số vòng lặp nhưng lại có chi phí tính toán mỗi vòng lặp cao, chỉ khả thi với các bài toán có số chiều nhỏ. Phần thực nghiệm sau đây sẽ minh họa rõ nét sự đánh đổi này.

4 Kết quả thực nghiệm và phân tích

4.1 Thí nghiệm 1: So sánh tốc độ hội tụ cơ bản



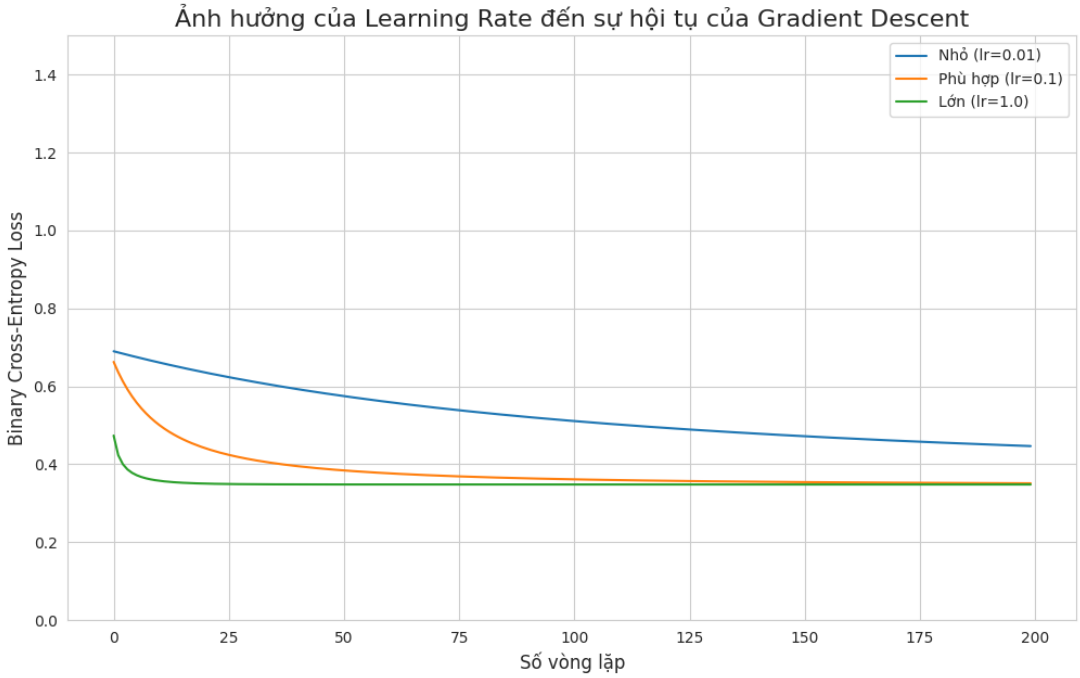
Hình 2: So sánh tốc độ hội tụ của GD ($\eta = 0.1$) và Newton.

Bảng 2: So sánh hiệu năng cơ bản.

Chỉ số	**Gradient Descent**	**Phương pháp Newton**
Số vòng lặp để hội tụ	~1000	7
Thời gian huấn luyện (giây)	0.1016	0.0142
Độ chính xác (Accuracy)	85.25%	85.25%

Nhận xét: Kết quả xác thực mạnh mẽ lý thuyết. Phương pháp Newton hội tụ chỉ sau 7 vòng lặp, trong khi GD cần hàng trăm vòng lặp mới tiệm cận được giá trị loss tương đương.

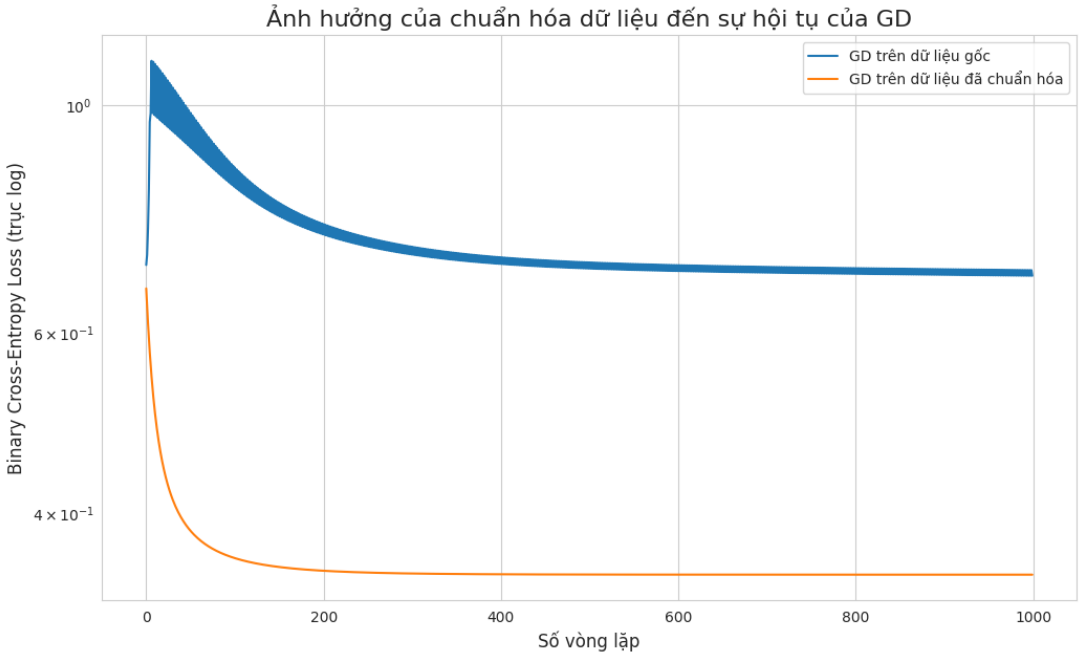
4.2 Thí nghiệm 2: Khảo sát sự nhạy cảm của GD với Learning Rate



Hình 3: Ảnh hưởng của learning rate đến sự hội tụ của Gradient Descent.

Nhận xét: Hình 3 cho thấy việc lựa chọn learning rate là tối quan trọng đối với các thuật toán bậc một. Nếu η quá nhỏ, thuật toán hội tụ rất chậm. Nếu η quá lớn, thuật toán có thể không ổn định và phân kỳ.

4.3 Thí nghiệm 3: Khảo sát ảnh hưởng của chuẩn hóa dữ liệu



Hình 4: So sánh tốc độ hội tụ của GD trên dữ liệu gốc và dữ liệu đã chuẩn hóa.

Nhận xét: Hình 4 cho thấy GD hội tụ **nhanh hơn và mượt mà hơn rất nhiều** trên dữ liệu đã được chuẩn hóa. Điều này là do việc chuẩn hóa giúp cải thiện số điều kiện của bài toán, làm cho bề mặt hàm loss "tròn"hơn và giảm hiện tượng "zig-zag"của GD.

5 Kết luận

5.1 Kết quả đạt được

Đồ án này đã thực hiện một bài khảo sát thực nghiệm có hệ thống nhằm phân tích và đối chiếu hai họ thuật toán tối ưu kinh điển – phương pháp bậc một (Batch Gradient Descent) và phương pháp bậc hai (Phương pháp Newton) – trong bối cảnh cụ thể của bài toán tối ưu lồi là Hồi quy Logistic. Bằng cách sử dụng bộ dữ liệu thực tiễn Heart Disease UCI làm "phòng thí nghiệm", đồ án đã thành công trong việc kết nối giữa lý thuyết tối ưu và hiệu năng thực nghiệm.

Các kết quả chính đạt được bao gồm:

- **Xác thực bằng thực nghiệm các đặc tính hội tụ lý thuyết:** Thông qua việc phân tích đường cong mất mát, đồ án đã minh họa một cách trực quan sự khác biệt cốt lõi giữa tốc độ hội tụ tuyến tính của Gradient Descent và tốc độ hội tụ bậc hai của Phương pháp Newton.

- **Lượng hóa các yếu tố ảnh hưởng đến hiệu năng của Gradient Descent:** Các thí nghiệm được thiết kế chuyên biệt đã chứng minh một cách định lượng rằng, không giống như Phương pháp Newton, hiệu quả của Gradient Descent phụ thuộc rất nhiều vào hai yếu tố ngoại sinh: (1) việc lựa chọn siêu tham số *learning rate* và

(2) sự cần thiết của việc tiền xử lý dữ liệu (chuẩn hóa) để cải thiện số điều kiện của bài toán.

Những kết quả này không chỉ đơn thuần là các con số đo lường hiệu năng, mà còn là nền tảng vững chắc để rút ra các kết luận sâu sắc hơn về bản chất và bối cảnh ứng dụng của từng phương pháp, được trình bày chi tiết trong phần tiếp theo.

5.2 Kết luận và Ý nghĩa thực tiễn

Dựa trên phân tích kết quả thực nghiệm, báo cáo đúc kết ba nhận định cốt lõi về việc áp dụng các thuật toán tối ưu trong mô hình Hồi quy Logistic:

1. Đặc tính hội tụ của bài toán tối ưu lồi Thực nghiệm cho thấy Gradient Descent và Phương pháp Newton đều hội tụ về cùng một nghiệm tối ưu (độ chính xác tương đồng 85.25%). Điều này kiểm chứng tính chất lý thuyết quan trọng của hàm mất mát lồi: *mọi cực trị địa phương đều là cực trị toàn cục*. Do đó, đối với các bài toán lồi, tiêu chí lựa chọn thuật toán không phụ thuộc vào chất lượng nghiệm cuối cùng, mà được quyết định bởi chi phí tính toán và tốc độ hội tụ phù hợp với tài nguyên hệ thống.

2. Phạm vi ứng dụng của phương pháp bậc hai (Newton's Method) Mặc dù lý thuyết chỉ ra chi phí tính toán ma trận Hessian là lớn ($\mathcal{O}(n^3)$), nhưng thực nghiệm trên bộ dữ liệu **Heart Disease** ($N = 303, d = 13$) chứng minh rằng rào cản này không đáng kể đối với các bài toán quy mô vừa và nhỏ. Trong các ứng dụng y sinh hoặc kinh tế lượng với số chiều dữ liệu không quá lớn, Phương pháp Newton thể hiện ưu thế vượt trội so với các phương pháp bậc một nhờ tốc độ hội tụ bậc hai (quadratic convergence), giảm thiểu đáng kể số vòng lặp và thời gian huấn luyện thực tế.

3. Vai trò tiên quyết của chuẩn hóa dữ liệu (Feature Scaling) Kết quả so sánh định lượng khẳng định sự phụ thuộc lớn của các phương pháp tối ưu bậc một (như Gradient Descent) vào *số điều kiện* (condition number) của ma trận Hessian. Trên dữ liệu thô chưa chuẩn hóa, thuật toán hội tụ rất chậm do quỹ đạo tối ưu bị dao động mạnh. Vì vậy, chuẩn hóa dữ liệu (Data Normalization/Standardization) không chỉ là bước cải thiện hiệu năng bổ trợ, mà là yêu cầu kỹ thuật bắt buộc để đảm bảo tính ổn định và khả năng hội tụ hiệu quả của thuật toán Gradient Descent.

5.3 Hạn chế

Mặc dù đã đạt được các mục tiêu đề ra, đồ án này vẫn có một số hạn chế nhất định:

- **Phạm vi thuật toán:** Đồ án chỉ tập trung vào hai thuật toán cơ bản nhất của hai họ. Một bài khảo sát toàn diện hơn có thể bao gồm các phương pháp Quasi-Newton (như L-BFGS) và các thuật toán tối ưu ngẫu nhiên phổ biến (như Adam, RMSProp).
- **Phạm vi bài toán:** Thực nghiệm chỉ được tiến hành trên một bài toán tối ưu lồi (Hồi quy Logistic). Hành vi và hiệu năng của các thuật toán này có thể thay đổi đáng kể trên các bài toán có hàm mục tiêu không lồi.

Tài liệu

[1] Andrew Ng. *CS229 Lecture Notes: Classification and Logistic Regression*. Computer Science Department, Stanford University.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. (Chapter 4: Linear Models for Classification).

[3] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. (Chapter 9: Unconstrained Minimization).

[4] Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). *Heart Disease Data Set*. UCI Machine Learning Repository. `https://archive.ics.uci.edu/ml/datasets/heart+disease`

[5] Scikit-learn Developers. *Logistic Regression - Scikit-learn User Guide*. `https://scikit-learn.org/stable/modules/linear_model.html`